

How an Approved Candidate Reaches main

Understanding candidate *C*, landing *L*, Git tree objects, and the watcher operations that connect them

Purpose	Explain the landing mechanism for a reader who normally uses commits, branches, merges, and pull requests.	Audience	Students, users, and maintainers of the AI-assisted coding ticket system.
Key rule	The Architect approves exact candidate <i>C</i> . The watcher calculates, verifies, and records distinct landing <i>L</i> .	Scope	Git objects, the three central commands, safe local landing, recovery, and remote publication.

Executive summary

The Implementer creates immutable candidate commit *C*. The Architect audits that exact commit and either approves or rejects it. After approval, the watcher—the running `mailbox_daemon.py` program—combines the approved candidate change with current local `main`, creates distinct single-parent landing commit *L*, and verifies its Git objects and safe checkout conditions. The watcher then fast-forwards local `main`. It attempts one ordinary, non-force push and records a recovery instruction if the remote cannot be verified.

These are internal operations

The command fragments below explain what the watcher does. A user does not run them to land a ticket. The watcher owns landing, recovery records, and the safe update of `main`.

1 The five objects to keep separate

<i>B</i>	The saved version from which the ticket began.
<i>C</i>	The exact candidate commit produced by the Implementer and audited by the Architect.
<i>M</i>	The exact commit at the tip of local <code>main</code> when landing is prepared.
<i>T</i>	A Git <i>tree object</i> : the complete repository snapshot obtained by combining <i>M</i> with the approved change represented by <i>C</i> .
<i>L</i>	The new landing commit. It points to snapshot <i>T</i> and has <i>M</i> as its sole parent.

One-sentence mental model

C is what the Architect approved; *T* is the integrated repository snapshot; *L* records that snapshot as the next commit after current local `main`.

2 Why the word “tree” is confusing

Git uses **tree** in a narrower sense than ordinary conversation. Three related ideas are often mixed together:

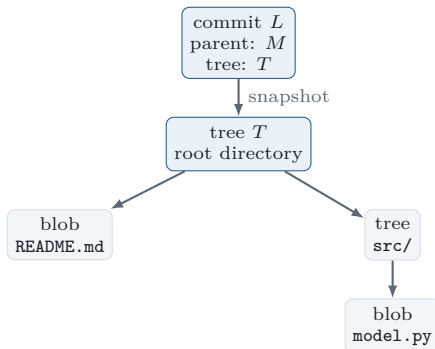
1. A **directory tree**: folders and files on disk.
2. A **commit graph**: commits connected by parent relationships.
3. A Git **tree object**: Git’s stored description of one direc-

tory snapshot.

In `git commit-tree` and `git merge-tree`, the important meaning is number 3.

2.1 A Git commit does not directly contain files

A commit points to a root tree object. That tree lists filenames and points to either file contents, called **blobs**, or more tree objects that represent subdirectories.



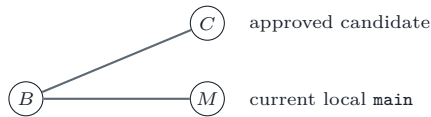
The root tree *T* means:

“Here is the complete set of filenames, directories, permissions, and file contents for this version of the repository.”

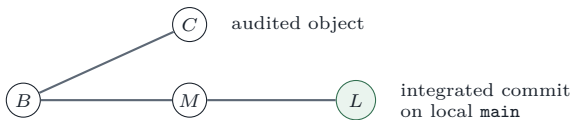
It does *not* mean a branch, a checked-out folder, or a history graph.

3 Candidate *C* versus landing *L*

Suppose the candidate started from *B*. While it was being developed, another accepted ticket advanced local `main` to *M*:



The watcher must preserve all accepted work already in *M* and the approved change from the *B* → *C* side. It creates a linear commit *L* after *M*:



C and *L* are distinct commits. A commit hash depends on its parent, message, author and committer metadata, timestamp, and root tree. *L* has parent *M* and a recovery-bound message, so the watcher rejects any record that tries to use *C* itself as *L*.

4 The three central Git commands

The following shell-shaped transcription makes the data flow visible. The Python watcher passes the landing message to `commit-tree` directly rather than launching this shell pipeline.

```
M=$(git rev-parse --verify \
  'refs/heads/main~{commit}')

T=$(git merge-tree --write-tree "$M" "$C")

L=$(printf '%s' "$LANDING_MESSAGE" | \
  git commit-tree "$T" -p "$M" -F -)
```

The notation `X=$(command)` means: run `command`, capture the text it prints, and store that text in variable `X`.

4.1 Command 1: freeze current local main

```
M=$(git rev-parse --verify \
  'refs/heads/main~{commit}')
```

`refs/heads/main` is Git's full internal name for the local branch `main`. A branch name is movable; it points to a commit. The `{commit}` suffix requires that the result be a commit, and `-verify` refuses an invalid name.

The command asks: "What exact commit does local `main` point to?" It may print:

```
a7f83011c38cb4e91e453a137efab5f727d91631
```

That immutable hash is stored in *M*. No file, branch, or commit changes.

Why record *M*?

`main` is a moving label. *M* is a fixed object. The landing operation can prove exactly which version it used as the parent.

4.2 Command 2: compute the integrated snapshot

```
T=$(git merge-tree --write-tree "$M" "$C")
```

`git merge-tree` invokes Git's three-way merge machinery. Git finds a common ancestor and compares both sides:

$B \rightarrow M$	changes already on main,
$B \rightarrow C$	approved candidate changes.

`-write-tree` stores the successfully combined repository snapshot as a new tree object and prints its hash. At this point Git has created only a snapshot object. It has not created a commit, moved a branch, staged a file, or modified a checkout.

This still uses Git's merge algorithm

The workflow avoids a conventional checked-out `git merge` and a two-parent merge commit. It does not avoid merge logic. `merge-tree` calculates the integrated file contents.

4.3 What happens on a conflict?

Suppose both sides change the same line incompatibly:

```
# At common base B
learning_rate = 0.001

# At current local main M
learning_rate = 0.0005

# At candidate C
learning_rate = 0.01
```

Git cannot determine which value is correct. `merge-tree` reports failure, and the watcher stops.

Why stopping is correct

Automatic conflict resolution would create code different from candidate *C* that the Architect audited. The conflict requires new implementation and a new audit rather than an unreviewed repair during landing.

4.4 Command 3: create the landing commit

```
L=$(printf '%s' "$LANDING_MESSAGE" | \
  git commit-tree "$T" -p "$M" -F -)
```

`git commit-tree` creates a commit directly from Git objects:

<code>\$T</code>	The complete repository snapshot for the new commit.
<code>-p \$M</code>	The new commit's sole parent is <i>M</i> .
<code>-F -</code>	Read the complete commit message from standard input.

The watcher does not invent a generic landing message. It copies the complete human-readable message already stored in approved candidate *C*, then appends two machine-readable recovery lines:

```
Mailbox-Cycle: EXACT-CYCLE-ID
Mailbox-Candidate: FULL-CANDIDATE-COMMIT
```

Those lines bind *L* to the ticket cycle and exact candidate that the Architect approved. `commit-tree` prints the new commit hash, which is stored in *L*.

Why *L* is not a merge commit

A merge commit has two or more parents. *L* has exactly one parent, *M*. Its snapshot contains the integrated files, but the history of `main` remains linear.

5 Why no checkout or staging is needed

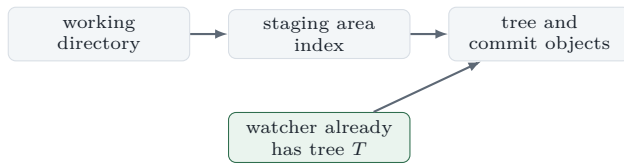
Most everyday Git commands operate through three layers:

1. the working directory;
2. the staging area, also called the index;
3. the Git object database.

The familiar sequence

```
git add file.py
git commit -m "message"
```

reads files from the working directory, places their desired state in the index, and then creates tree and commit objects. The watcher already knows exact tree T , so it can create L without using the working directory or index:



Constructing L therefore does not touch the user's files. Updating the user's folder is a later, separately checked step.

6 A complete worked example

Assume common base B contains:

```
src/model.py
tests/test_model.py
```

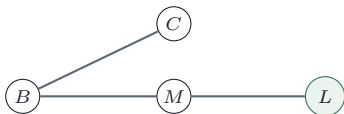
Candidate C modifies both files. Another accepted ticket advances main to M by adding:

```
src/logging.py
```

`merge-tree` calculates T with all three desired files:

```
src/logging.py      # retained from M
src/model.py        # candidate version from C
tests/test_model.py # candidate version from C
```

`commit-tree` records that snapshot as L with parent M :



`git diff M L` contains only the approved candidate delta. The `logging.py` addition is already present in both M and L , so it does not appear as a new change in that comparison.

7 Why not use ordinary merge or cherry-pick?

7.1 Ordinary merge

A conventional merge may create a commit with both M and C as parents. That would preserve the candidate branch topology in main . This workflow wants one linear landing commit for one accepted ticket.

7.2 Cherry-pick

Cherry-pick naturally applies one or more commits. Candidate development may contain several intermediate commits:

$$B \rightarrow c_1 \rightarrow c_2 \rightarrow C$$

The landing rule records their net result as one commit after M . `merge-tree` plus `commit-tree` computes the complete branch result without replaying each intermediate implementation commit into main .

8 How the watcher verifies and records landing L

Creating L does not immediately move main . The watcher first:

1. saves L under a private Git reference and checks that the reference points to exact L ;
2. checks that L has exactly one parent, M ;
3. recalculates the expected squash tree and compares it with L 's tree; and
4. checks that L 's message exactly preserves C 's approved message and the two recovery lines.

These are Git identity and recovery checks. They do *not* rerun the ticket's scientific tests or gates on L . The Architect already audited the test evidence for exact candidate C .

Why the private reference matters

A new Git object can become hard to recover if no branch or reference points to it. The private landing reference preserves exact L before the user's branch changes. A restart verifies and reuses that object instead of silently building a different landing.

9 When does local main move?

The user's main folder must pass every condition below:

- Git has the folder attached to local branch main ;
- local main still points to parent M used to build L ; and
- the folder has no staged, unstaged, or untracked files.

Only then does the watcher run the equivalent of:

```
git merge --ff-only L
```

Because L has M as its direct parent, this command performs no new content merge. It moves the branch and clean checkout from M to L .

If the folder is not safe

The watcher leaves the user's folder untouched and preserves candidate C , Architect GO, and prepared landing L . Unfinished files require the user to make the folder clean and restart. If `main` moved to another saved commit after L was prepared, candidate C requires a new Architect audit.

10 Local landing and GitHub are separate

Local `main` reaching L completes the local landing. The watcher then attempts one ordinary, non-force push of exact L to remote `origin/main` and checks that the remote reports that same commit.

If the push fails or the remote result cannot be verified, the watcher does not rewrite history and does not repeat the ticket. Local L remains accepted. A file named `pending-main-push-L.txt` records the exact commit and command still owed.

Two separate questions

"Did the ticket land locally?" asks whether local `main` is exact L . "Did GitHub receive it?" asks whether `origin/main` is exact L . A network or remote-branch problem can make the second answer no while the first answer remains yes.

11 Safety properties

- **Immutable audit target:** the Architect's decision names exact candidate C .
- **Distinct accepted object:** the watcher rejects any landing record that uses C itself as L .
- **Current-main preservation:** L is built after actual landing parent M .
- **Fail-closed conflicts:** incompatible concurrent changes stop landing.
- **Linear main history:** L has one parent.
- **No role-owned integration:** the parent watcher, not Architect, Implementer, or Red Team, creates L and updates `main`.
- **No hidden test claim:** object verification is not described as a new scientific test run.
- **Recoverable preparation:** a private reference preserves exact L before the branch moves.
- **Protected remote history:** the watcher attempts no force push.

12 Where the implementation lives

The current source of truth is `ai/tools/mailbox_daemon.py`. These functions divide the mechanism into reviewable steps:

Function	Responsibility
<code>_exact_squash_tree</code>	Calculate T with <code>merge-tree</code> .
<code>_landing_commit_message</code>	Preserve C 's message and add recovery facts.
<code>prepare_exact_squash_landing</code>	Create, journal, and verify L .
<code>_verify_prepared_landing</code>	Recalculate parent, tree, and message checks.
<code>land_prepared_commit_in_clean_user_checkout</code>	Check the user's folder and fast-forward local <code>main</code> .
<code>push_exact_landing_or_record_debt</code>	Push once without force or save the owed action.

13 Compact translation table

Command or object	Plain-English meaning
<code>C</code>	Exact implementation approved by the Architect.
<code>M</code>	Exact current commit at local <code>main</code> .
<code>T</code>	Complete integrated snapshot stored as a Git tree object.
<code>L</code>	Distinct single-parent commit that records T after M .
<code>rev-parse</code>	Resolve and verify a name as one exact commit.
<code>merge-tree</code>	Calculate merged file contents without a checkout-based merge.
<code>-write-tree</code>	Store the calculated snapshot and print its hash.
<code>commit-tree</code>	Create a commit directly from a tree object.
<code>-p M</code>	Declare M as the new commit's parent.
<code>-F -</code>	Read the complete commit message from standard input.
<code>-ff-only</code>	Move <code>main</code> only when no new merge is required.

Final mental model

1. Freeze current local `main` as commit M .
2. Calculate integrated snapshot T from M and approved candidate C .
3. Create distinct landing L with snapshot T and sole parent M .
4. Verify L 's parent, tree, message, and recovery reference.
5. Check the user's `main` folder and fast-forward it from M to L .
6. Attempt one non-force push; record the exact owed command if the remote cannot be verified.